Technical Design Note: BM25 and Embedding News Recommendation

Arnav Agnihotri

August 2026

1 System Built

The project implements a reproducible offline news recommendation pipeline for MIND-small and EB-NeRD-demo. Raw MIND TSV files and EB-NeRD Parquet files are normalized into the same three-table schema: `articles` for metadata and text, `history` for click events, and `impressions` for candidate-label events. The core fields are article/user/impression IDs, dataset, title, subtitle, category, timestamps, label, and split. This lets the same retrieval, ranking, evaluation, and submission code run on both datasets.

Architecture. The end-to-end flow is: raw archive extraction → dataset parser → unified Parquet tables → cached BM25/embedding indexes → validation and test candidate scoring → offline metrics and Codabench-format predictions. MIND articles use `title + subtitle`; MIND has no per-click timestamps, so historical clicks are assigned a conservative proxy `click_time = impression_time - 1s`. EB-NeRD uses its real click timestamp arrays from `history.parquet`; article, user, and impression IDs are converted to strings for a shared schema.

Retrieval and candidate generation. The implementation has two complementary rankers rather than a learned hybrid fusion model. For recall experiments, each ranker retrieves top-K articles from the full article corpus for sampled validation impressions. For ranking/evaluation/submission, each ranker reranks the candidate list already supplied by the impression. This distinction is important: full-corpus retrieval measures whether the clicked article is reachable; impression reranking measures ordering quality among articles the platform already exposed.

User and news representations. A news item is represented by its `title + subtitle`. The BM25 index tokenizes this text with a simple lowercase regex tokenizer over word tokens of length at least two. A user query is built from the most recent 50 clicked articles strictly before the impression time; the concatenated query is capped at 100 tokens for speed. The embedding ranker uses 384-dimensional `all-MiniLM-L6-v2` vectors for MIND and 300-dimensional pre-computed Ekstra Bladet Word2Vec document vectors for EB-NeRD. A user vector is the L2-normalized mean of up to 50 prior clicked article vectors, and candidate scores are cosine similarities, implemented as dot products after normalization.

Ranking. BM25 full-corpus recall uses `BM25Okapi.get_scores()` and sorts the whole corpus. Candidate reranking uses a custom per-document BM25 scorer, reducing cost from $O(|q|N)$ to $O(|q|C)$ where N is corpus size and C is the impression candidate count. Embedding recall computes `matrix @ user_vector` and uses `argpartition`; candidate reranking computes one dot product per candidate. Cold users, missing histories, and missing candidate embeddings receive score 0.0, so ties fall back to original order through stable rank assignment.

2 Design Choices

Why BM25 plus embeddings. BM25 is a strong lexical baseline for news because titles contain named entities, locations, and event terms where exact matches matter. It is cheap, explainable, and requires no training. Embeddings complement this by recovering semantic neighbors when the user and candidate article use different words, and by using external/precomputed representations where available. The measured results show this complementarity clearly on MIND: embeddings substantially improve AUC, MRR, and nDCG, while BM25 produces slightly higher novelty and diversity.

Temporal safety over convenience. Histories are stored broadly, but every feature builder filters `click_time < impression_time`. This avoids future-click leakage during validation. The leakage test samples impressions and verifies the exact windowing condition. For MIND the timestamp proxy is a limitation, but it preserves the ordering contract needed by the rankers.

Engineering trade-offs. The shared schema reduces duplicated dataset-specific code, but it loses some source-specific richness: EB-NeRD read time and scroll percentage are not used in the main validation ranker, and MIND entity annotations are ignored. Mean pooling is simple and robust but cannot model session order, topic drift, or multiple user interests. Caching BM25 pickles and compressed embedding matrices makes iteration practical; the trade-off is larger disk artifacts and index invalidation when article text or representation choices change.

3 Measured Observations

Processed validation scale and cache sizes are shown below. These counts were measured from the generated Parquet and cache artifacts in the project directory.

Dataset	Articles	History rows	Val imps	Val rows	Avg. cand.	Index cache
MIND	65,238	2,012,565	36,576	1,338,100	36.58	79.84 MB BM25 / 93.12 MB emb.
EB-NeRD-demo	11,777	295,851	12,678	149,941	11.83	38.91 MB BM25 / 13.14 MB emb.

Recall@K is measured on sampled validation impressions using full-corpus retrieval; BM25 uses the script default sample of 200 unless rerun, while embeddings use the default sample of 500 unless rerun. Ranking metrics are measured on the full validation candidate predictions.

Dataset	Ranker	R@50	R@100	R@200	AUC	MRR	nDCG@5	nDCG@10
MIND	BM25	0.0050	0.0125	0.0292	0.5229	0.2911	0.2672	0.3285
MIND	Emb.	0.0206	0.0234	0.0356	0.6524	0.3721	0.3585	0.4158
EB-NeRD	BM25	0.0050	0.0100	0.0200	0.4889	0.3111	0.3398	0.4271
EB-NeRD	Emb.	0.0080	0.0220	0.0380	0.4958	0.3116	0.3418	0.4293

On MIND, embeddings outperform BM25 by 0.1295 AUC, 0.0810 MRR, and 0.0874 nDCG@10. This is a measured result and supports the inference that semantic matching is useful for English title/abstract-like recommendation. On EB-NeRD-demo, the embedding improvement is much smaller: AUC rises from 0.4889 to 0.4958 and nDCG@10 from 0.4271 to 0.4293. This may reflect the lower-dimensional Word2Vec representation, Danish vocabulary/domain effects, missing vectors, or the fact that the candidate set already contains near-neighbor articles; the project does not isolate those causes experimentally.

Beyond-accuracy metrics show another trade-off. MIND BM25 has novelty 15.638 and ILD 0.879, while MIND embeddings have novelty 15.065 and ILD 0.833; embeddings are more accurate but less diverse by category. EB-NeRD has higher catalog coverage (about 0.15) than MIND (about 0.035), largely because the demo catalog is smaller. Cold/warm slicing uses a threshold of at most five historical clicks. MIND embeddings improve warm users more strongly (AUC 0.6647) than cold users (0.5961), consistent with mean-pooled histories needing enough clicks. EB-NeRD embeddings are unusually better on the cold slice by AUC (0.5683 vs. BM25 0.4409), but that slice has very low coverage and should be treated cautiously.

4 Failure Modes and Limits

Full-corpus recall remains low for both approaches: even embedding R@200 is only 0.0356 on MIND and 0.0380 on EB-NeRD-demo. This indicates that clicked articles are often not near the historical profile under either lexical overlap or mean-pooled semantic similarity. News recommendation is strongly driven by freshness, popularity, geography, editorial placement, and session intent; none of those signals are modeled in the main rankers.

BM25 fails when the candidate is topically related but uses different words, when histories are short, or when user interests are broad enough that concatenating many clicks creates a noisy query. Embeddings fail when mean pooling collapses multiple interests into one centroid, when all prior clicks are stale, or when EB-NeRD articles lack precomputed Word2Vec vectors. Both rankers degrade for cold users because the default score becomes 0.0. Data issues also matter: MIND lacks true historical click times, MIND raw impression IDs repeat across train/dev unless split context is included, and EB-NeRD contains richer engagement signals that the main validation pipeline leaves unused.

5 Scaling by 10x

At 10x scale, the bottlenecks move from modeling to indexing, memory, and latency. BM25 full-corpus scoring scans every document per query and becomes expensive for recall or open retrieval; the cached Python object is also not a production inverted index. Embedding recall stores an $N \times D$ float matrix and computes dense dot products; this is acceptable for tens of thousands of articles but strains memory bandwidth and latency at millions of articles and many concurrent requests. Exploding impressions into one row per candidate also grows quickly; the MIND-large script already bypasses Pandas because 2.37M impressions times roughly 50 candidates would create over 100M rows.

Recommended 10x Changes

The most important architectural changes are: (1) replace `rank_bm25` scans with a real inverted index such as Lucene/OpenSearch and fetch only a lexical top-K; (2) use FAISS or another ANN service for embedding retrieval with sharded, memory-mapped vector indexes; (3) introduce a two-stage architecture that unions BM25, ANN, popularity, and recency candidates, then reranks a few hundred items with calibrated features; (4) store user profiles incrementally in a feature store instead of recomputing histories from Parquet for every batch; and (5) stream candidate scoring and submission output, keeping impressions nested until scoring time rather than materializing massive exploded tables.